

Agent Execution Ledger (AEL) 설계 문서

Message Notification Router 프로젝트 — 에이전트 의사결정을 감사 가능하게 기록하는 Single Source of Truth (SSOT) 구조.

1. AEL이란 무엇인가

AEL(Agent Execution Ledger)은 단순한 실행 로그가 아니라, 에이전트가 하나의 작업을 처리하는 전 과정 — 계획(Planning) → 실행(Execution) → 근거(Evidence) → 검증(Verification) → 반성(Reflection) → 상태(State) — 를 하나의 append-only 기록으로 남기는 문서/데이터 구조다. 목적은 두 가지다.

- 설명 가능성: “왜 이 메시지를 notify로 분류했는가”를 사후에 정확히 역추적할 수 있다.
- 환각 방지: “에이전트가 했다고 주장하는 것”과 “실제로 검증을 통과한 것”을 구조적으로 분리한다 — Verification 없이는 State가 갱신되지 않는다.

2. 6단계 파이프라인

단계	질문	이 프로젝트에서의 의미
Planning	무엇을 하려는가?	이 메시지를 notify/digest/mute 중 무엇으로 분류할지 결정하기 위해 pull_context()가 어떤 컨텍스트(user/group/business/media/history)를 모을지 계획.
Execution	실제로 뭘 했는가?	classify_message()가 JSON 스키마로 제약된 Claude 호출을 실행 — 모델, 토큰 사용량, 성공/실패 상태를 기록.
Evidence	그 결정의 근거는?	evidence_message_ids로 인용된 과거 메시지, 첨부 이미지/음성 분석 결과 — 실제로 사용된 근거만 기록.
Verification	근거가 결정을 충족하는가?	action이 허용된 값인지, evidence_message_ids가 실제 존재하는 message_id인지, confidence가 얼마인지 자동 검증.
Reflection	왜 이렇게 됐고, 다음은?	router.py가 이미 계산하는 내부 판단 필드 (sender_trust, urgency_signal, risk_signal, repetition_signal) 그대로 재사용 — 실패 시 next_action=flag_for_retry로 표시.
State	지금 무엇을 알고 있는가?	Planning~Reflection 원본 기록은 그대로 두고, 최종 action/message_type/confidence/evidence를 압축한 체크포인트 — output.csv 한 행과 1:1 대응.

3. 왜 이 구조가 SSOT인가

- Append-only: 어떤 레코드도 수정하지 않는다. 재처리(retry-failed)는 새 State 레코드를 추가할 뿐, 이전 기록을 지우지 않는다.
- FK 체인으로 역추적: State → Reflection → Verification → Evidence → Execution → Planning 순으로 message_id 하나의 전체 이력을 재구성할 수 있다.
- 기존 파이프라인과 손실 없이 통합: router.py의 classify_message()가 이미 만들어내는 reason/confidence/evidence_message_ids/internal 필드를 그대로 기록하므로, 프롬프트나 라우팅 로직을 바꾸지 않고 감사 계층만 추가한다.

- data/processed.db와 분리: db.py는 매 실행마다 processed.db를 dataset/*.csv로부터 새로 만든다(원본 데이터의 읽기 전용 캐시). AEL은 별도의 data/acl.db에 저장되어 그 재빌드에 영향받지 않는다 — 에이전트가 실제로 한 일의 기록이지, 입력 데이터의 사본이 아니기 때문이다.

4. 구현: ael_schema.sql + ael_client.py

code/acl_schema.sql은 6단계에 대응하는 6개 테이블(acl_planning, acl_execution, acl_evidence, acl_verification, acl_reflection, acl_state)을 정의한다. 모든 테이블은 message_id로 색인되어 있어 특정 메시지의 전체 판단 과정을 바로 조회할 수 있다.

code/acl_client.py는 이 스키마에 대한 얇은 파이썬 래퍼다 — router.classify_message()가 반환하는 record 딕셔너리와 pull_context()가 반환하는 ctx 딕셔너리를 받아 record_cycle() 한 번 호출로 6단계를 모두 기록한다:

```
from acl_client import init_db, record_cycle

init_db() # ■■■ 1■: data/acl.db ■■■

# router.process_message() ■■■, classify_message() ■■ ■■
ctx = pull_context(conn, message_row)
record = classify_message(message_row, conn, client)
record_cycle(message_row, ctx, record)
```

조회는 get_cycle(message_id)로 Planning부터 State까지 한 번에 재구성할 수 있다 — 낮은 confidence로 분류된 메시지를 나중에 검토할 때, 어떤 근거와 내부 신호로 그 결정에 도달했는지 바로 확인 가능하다.

5. 다음 단계 (선택)

- router.py의 process_message() 안에 record_cycle() 호출을 실제로 연결.
- cmd_cache_status에 AEL 기반 통계(예: next_action=flag_for_retry 건수, 평균 confidence) 추가.
- 심사 발표 자료에서 특정 message_id 하나를 골라 get_cycle()로 전체 추론 과정을 시연 — “black box가 아니다”를 코드로 증명하는 가장 직접적인 방법.